``

Chapter 2: Interview AI Service

Welcome back! In our previous chapter, we learned about the "Frontend Interview Management." We saw how the buttons and forms you interact with on your screen send requests to the "brain" of our application. Now, it's time to actually open up that "brain" and see how it works!

What Problem Are We Solving? (The AI's Job!)

Imagine you've just submitted your resume and a job description to our app. The frontend has sent this information off. Now what? This is where the **Interview AI Service** comes in. It's the "smart part" of our application, like a super-intelligent interview coach working tirelessly behind the scenes.

Its main job is to take all that raw information (your resume, how you describe yourself, and the job description) and transform it into incredibly useful insights. This includes:

- **Generating smart interview questions:** Both technical and behavioral.
- **Spotting skill gaps:** "Hmm, this job requires 'cloud computing,' but your resume doesn't mention it much."
- **Creating a personalized preparation plan:** "Okay, let's focus on these areas for the next 5 days."
- **Even tailoring your resume PDF:** Making it perfect for that specific job.

Think of it as the ultimate personal assistant for job preparation, all powered by Artificial Intelligence!

Key Concepts: Talking to the AI

To make our AI coach work, we need a few special tools and ideas:

1. **Large Language Models (LLMs):** This is the core "AI brain" we use. We're leveraging Google's powerful **Gemini** model. Imagine it as a super-smart robot that can read, understand, and generate human-like text. It's really good at taking instructions and producing creative, structured answers.

2. **Prompt Engineering:** This is how we "talk" to the LLM. You don't just type "give me interview questions." You craft a very specific instruction, called a "prompt," telling the AI exactly what you want, what information it has, and how it should format its response. It's like giving clear instructions to a chef: "I need a 3-course meal, vegetarian, served on a round plate."
3. **Schema (Zod):** Sometimes, we need the AI to give us information in a very specific, organized way (like a list of questions, each with a 'question' and 'answer' field). A "schema" is like a blueprint that tells the AI: "Your answer *must* look like this." We use a library called **Zod** to define these blueprints. This ensures our application can easily understand and use the AI's output.
4. **PDF Parsing (pdf-parse):** Your resume is a PDF file. Before the AI can read it, our service needs to extract the text from that PDF. "PDF Parsing" is the process of converting the unreadable PDF format into plain text that the AI can understand.
5. **PDF Generation (Puppeteer):** After the AI suggests how to tailor a resume (as HTML code), we need to turn that HTML into a professional-looking PDF file. "PDF Generation" is the process of converting web-page code (HTML) into a downloadable PDF document, and we use a tool called **Puppeteer** for this.

How Our Backend Uses the Interview AI Service

Let's trace what happens when you click "Generate Interview Report" from the frontend.

In Chapter 1: Frontend Interview Management, we saw the generateInterviewReport function in interview.api.js send data to our backend. On the backend, a "controller" function receives this request.

Here's a simplified look at the generateInterViewReportController in Backend/src/controllers/interview.controller.js:

```
// Backend/src/controllers/interview.controller.js (simplified)
const pdfParse = require("pdf-parse"); // Tool to read PDFs
const { generateInterviewReport } = require("../services/ai.service"); // Our AI brain!

async function generateInterViewReportController(req, res) {
  // 1. Get resume text from the uploaded PDF file
  const resumeContent = await (new
pdfParse.PDFParse(Uint8Array.from(req.file.buffer))).getText();

  // 2. Get other inputs (self-description, job-description)
  const { selfDescription, jobDescription } = req.body;

  // 3. Ask the AI Service to generate the report!
```

```
    const aiReport = await generateInterviewReport({
        resume: resumeContent.text,
        selfDescription,
        jobDescription
    });

    // 4. Save the report to our database (explained in later chapters)
    // const interviewReport = await interviewReportModel.create({...});

    res.status(201).json({
        message: "Report generated!",
        interviewReport: aiReport // Sending AI's smart insights back!
    });
}
```

Explanation:
This generateInterViewReportController is the first point of contact on the backend for generating a report.

1. It uses pdfParse to read the text content directly from the uploaded resume PDF.
2. It takes your selfDescription and jobDescription that came from the frontend.
3. Crucially, it then calls generateInterviewReport from our ai.service.js (our "AI brain" service) and passes all this information to it.
4. Once the AI service does its magic and returns the aiReport, this controller sends that detailed report back to the frontend.

Similarly, for generating a custom resume PDF, the backend calls generateResumePdf from the same ai.service.js:

```
// Backend/src/controllers/interview.controller.js (simplified for PDF generation)
const { generateResumePdf } = require("../services/ai.service");

async function generateResumePdfController(req, res) {
    const { interviewReportId } = req.params;
    // (Imagine fetching the original resume, job, self-description from the saved report)
    const { resume, jobDescription, selfDescription } = /* ... fetched report ... */;

    // Ask the AI Service to generate the PDF!
    const pdfBuffer = await generateResumePdf({ resume, jobDescription, selfDescription });

    res.set({ /* ... PDF headers ... */ });
    res.send(pdfBuffer); // Send the actual PDF file back
}
```

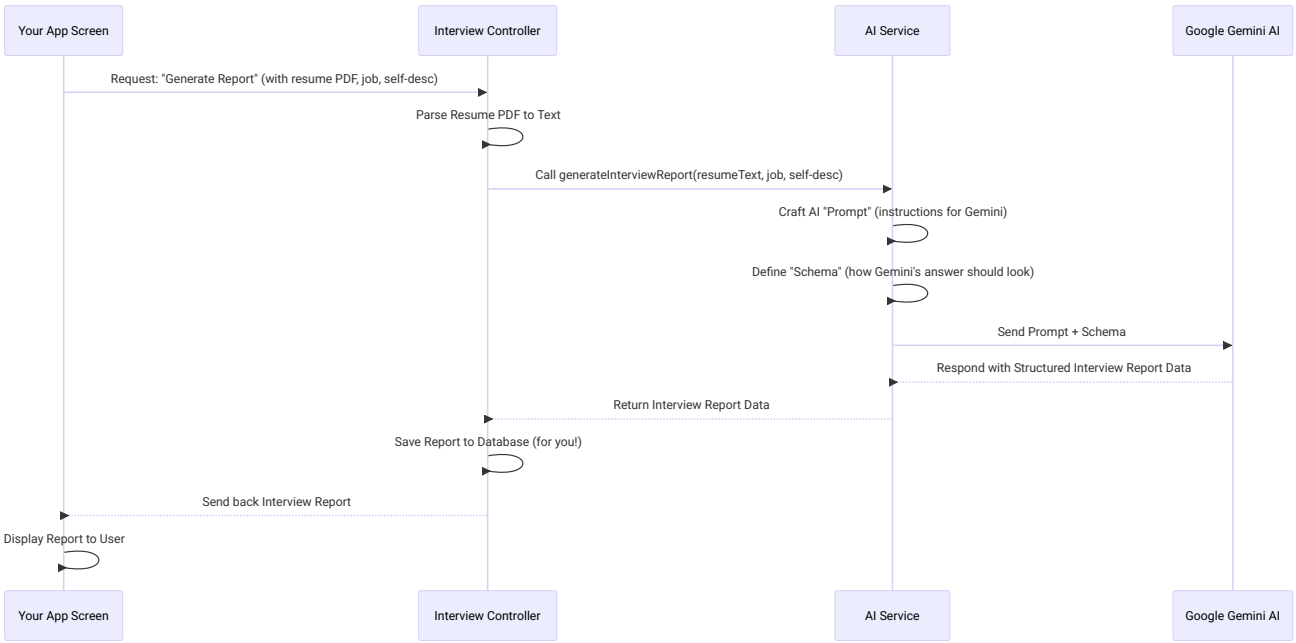
Explanation:

This controller fetches the necessary details (original resume text, job description, self-description) associated with a specific interview report. Then, it calls generateResumePdf in our AI service, which will produce a tailored resume in PDF format. The controller then sends this PDF directly back to the user.

Under the Hood: The AI Service's Magic!

Now, let's peek inside Backend/src/services/ai.service.js to see how our "AI brain" actually talks to the Gemini model and processes information.

Here's the overall flow for generating an interview report:



1. Crafting the AI Prompt and Schema

In ai.service.js, the most important part is how we ask the AI for information. We use a **prompt** (our detailed instructions) and a **schema** (our blueprint for the answer).

First, let's look at the interviewReportSchema using Zod. This tells the AI the exact structure we want for our interview report:

```
// Backend/src/services/ai.service.js (simplified Zod schema)
const { z } = require("zod"); // Zod library

const interviewReportSchema = z.object({
```

```
    matchScore: z.number().describe("Score 0-100"),
    technicalQuestions: z.array(z.object({
        question: z.string(),
        intention: z.string(),
        answer: z.string()
    })),
    // ... many more fields for skillGaps, preparationPlan, etc. ...
    title: z.string().describe("The title of the job"),
});
```

Explanation:
This interviewReportSchema is a clear instruction to the AI. It says: "Hey, your report *must* be an object that has a matchScore (which is a number), technicalQuestions (which is a list of objects, and each object *must* have a question, intention, and answer), and so on." This ensures the AI's output is always predictable and easy for our application to work with.

2. Talking to Gemini for an Interview Report

Now, let's see how generateInterviewReport uses this schema and a prompt to get the report from Gemini:

```
// Backend/src/services/ai.service.js (simplified generateInterviewReport function)
const { GoogleGenAI } = require("@google/genai"); // Gemini AI library
const { zodToJsonSchema } = require("zod-to-json-schema"); // Convert Zod to AI-friendly format

const ai = new GoogleGenAI({ /* ... API key ... */ }); // Connect to Google AI

async function generateInterviewReport({ resume, selfDescription, jobDescription }) {
    // 1. Our detailed instructions for the AI (the "prompt")
    const prompt = `Generate an interview report for a candidate with the following details:
                    Resume: ${resume}
                    Self Description: ${selfDescription}
                    Job Description: ${jobDescription}`;

    // 2. Send the prompt and our desired schema to Gemini
    const response = await ai.models.generateContent({
        model: "gemini-3-flash-preview", // The specific Gemini model we're using
        contents: prompt,
        config: {
            responseMimeType: "application/json", // We want JSON output
            responseSchema: zodToJsonSchema(interviewReportSchema), // Use our blueprint!
        }
    });
```

```
    // 3. Get the AI's answer and parse it
    return JSON.parse(response.text);
}
```

Explanation:

1. The prompt is created by simply inserting the resume, selfDescription, and jobDescription into a template. This tells Gemini all the necessary context.
2. We then call ai.models.generateContent, specifying which Gemini model to use (gemini-3-flash-preview).
3. Crucially, we include responseSchema: zodToJsonSchema(interviewReportSchema). This is the magic handshake! It tells Gemini: "Please generate your response *according to this exact structure*."
4. Gemini processes the prompt and returns a response, which we then JSON.parse to turn into a JavaScript object our application can easily use.

3. Generating a Tailored Resume PDF

The process for generating a resume PDF is similar, but with an extra step:

```
// Backend/src/services/ai.service.js (simplified generateResumePdf function)
const puppeteer = require("puppeteer"); // Tool to generate PDFs from HTML

async function generatePdfFromHtml(htmlContent) {
    const browser = await puppeteer.launch(); // Open a 'virtual' web browser
    const page = await browser.newPage();
    await page.setContent(htmlContent, { waitUntil: "networkidle0" }); // Load the AI's HTML

    const pdfBuffer = await page.pdf({ format: "A4" }); // "Print" to PDF
    await browser.close();
    return pdfBuffer;
}

async function generateResumePdf({ resume, selfDescription, jobDescription }) {
    const resumePdfSchema = z.object({
        html: z.string().describe("The HTML content of the resume")
    }); // We want the AI to give us HTML!

    const prompt = `Generate resume for a candidate with the following details:
                    Resume: ${resume}
                    Self Description: ${selfDescription}
                    Job Description: ${jobDescription}
```

```
        The response should be a JSON object with a single field "html"
        which contains the HTML content of the resume tailored for the job...`; // Detailed instructions

    const response = await ai.models.generateContent({
        model: "gemini-3-flash-preview",
        contents: prompt,
        config: {
            responseMimeType: "application/json",
            responseSchema: zodToJsonSchema(resumePdfSchema), // Tell AI to return HTML
        }
    });

    const jsonContent = JSON.parse(response.text);
    const pdfBuffer = await generatePdfFromHtml(jsonContent.html); // Convert AI's HTML to PDF

    return pdfBuffer;
}
```

Explanation:

1. We define a simple resumePdfSchema that tells Gemini: "Just give me a JSON object with an html field, and that field should contain the entire resume as HTML code."
2. The prompt here is very specific, asking the AI to tailor the resume for the job description and format it as HTML, keeping it professional and ATS-friendly.
3. After Gemini responds with the HTML, we then use the generatePdfFromHtml function. This function uses **Puppeteer** (a headless browser) to take that HTML code, "render" it like a webpage, and then "print" it into a PDF file (pdfBuffer).
4. Finally, this PDF pdfBuffer is sent back to the controller, which then sends it to the frontend for you to download!

Conclusion

In this chapter, we explored the "Interview AI Service," the true "brain" of our application. We learned how it uses powerful AI models like Google Gemini to process your raw data, understand your needs through careful "prompt engineering" and "schemas," and generate smart insights like interview reports and tailored resume PDFs. We also saw how essential tools like pdf-parse and puppeteer help handle file formats.

This service is where all the "AI magic" happens, turning complex data into actionable preparation tools. Next, we'll shift our focus from AI to security! How do we make sure only *you* can access *your* interview reports? That's what we'll cover in next chapter